

CENTRO UNIVERSITÁRIO DE BRASÍLIA — CEUB

Business Intelligence II — Projeto Final

Vendas, Logística e Satisfação no E-commerce Brasileiro

Uma solução de BI ponta a ponta sobre o dataset público Olist

Aluno: **Mateus Itibal**

Professor: MsC Pedro H. Mello

Dataset: Brazilian E-Commerce Public Dataset by Olist (Kaggle)

Apresentação final: 23/06/2026

Sumário

Sumário	2
1. Sumário de execução	4
1.1 O que foi construído	4
1.2 Principais achados (referência do dataset)	4
2. Definição do problema de negócio	6
2.1 Contexto	6
2.2 Problema de negócio	6
2.3 Stakeholders	6
2.4 Hipóteses iniciais	6
2.5 Critério de sucesso	6
3. KPIs e OKRs	8
3.1 Indicadores-chave (KPIs)	8
3.2 OKRs	8
4. Modelagem dimensional (star schema)	10
4.1 Por que dois fatos	10
4.2 Dimensões	10
4.3 Diagrama do modelo	11
4.4 Relacionamentos	11
5. Transformações em Power Query	13
5.1 Transformações por tabela	13
5.2 Checklist de qualidade aplicado	13
6. Medidas em DAX	15
7. Row-Level Security (RLS)	17
7.1 Como foi testado	17
8. Dashboard	18
8.1 Página 1 — Visão Executiva	18
8.2 Página 2 — Análise de Vendas e Produtos	18
8.3 Página 3 — Logística e Satisfação	18
9. Versionamento e repositório	19
9.1 Estrutura do repositório	19
9.2 Esboço do README	19
10. Pontos de revisão	20
10.1 Checklist final	20

10.2 Perguntas prováveis da banca	20
---	----

Como ler este documento

Este é o documento de projeto que acompanha o repositório no GitHub e a apresentação. O dataset escolhido é real e público (Olist, no Kaggle).

Os valores de baseline citados nos KPIs são as faixas conhecidas desse dataset e servem de referência; os números exatos são os que o dashboard exibe depois do modelo construído. Validem o dataset com o professor por e-mail até a Aula 1 (02/06), como pede o enunciado.

1. Sumário de execução

Este projeto constrói uma solução de Business Intelligence completa para uma operação de e-commerce, usando os dados reais da Olist — uma plataforma que conecta pequenos e médios vendedores aos grandes marketplaces brasileiros. O conjunto cobre cerca de 100 mil pedidos realizados entre setembro de 2016 e outubro de 2018, com informações de itens, pagamentos, avaliações, clientes, produtos e vendedores.

A pergunta que move o trabalho é direta: onde a operação está perdendo receita e satisfação do cliente, e o que dá para priorizar para melhorar. Para respondê-la, os dados brutos (oito tabelas em formato "flat") foram tratados no Power Query, reorganizados em um modelo estrela com dois fatos e quatro dimensões, e analisados por um conjunto de medidas em DAX. O resultado é um dashboard de três páginas que liga vendas, logística e avaliação dos clientes.

1.1 O que foi construído

- **Modelo de dados:** esquema em constelação com dois fatos (itens de pedido e pedidos) compartilhando dimensões conformadas de Cliente e Calendário, mais as dimensões Produto e Vendedor.
- **Métricas:** 17 medidas em DAX organizadas em uma tabela de medidas dedicada — desde receita e ticket médio até crescimento ano a ano e taxa de atraso na entrega.
- **Segurança:** três papéis de RLS que simulam diretoria, gerência regional e análise comercial.
- **Dashboard:** Visão Executiva, Análise de Vendas e Produtos, e Logística e Satisfação.

1.2 Principais achados (referência do dataset)

As faixas abaixo são o ponto de partida da análise — o que o negócio observa hoje e quer melhorar. Os números finais são lidos diretamente do dashboard.

Indicador	Baseline aprox.	Leitura
Receita em itens (GMV)	~R\$ 13,6 mi	Tamanho da operação no período.
Ticket médio por pedido	~R\$ 137	Espaço para vender mais por compra.

Indicador	Baseline aprox.	Leitura
Prazo médio de entrega	~12 dias	Alto para o padrão de e-commerce.
Taxa de atraso vs. estimativa	~8%	Atrasos puxam a nota para baixo.
Nota média de avaliação	~4,0 de 5	Boa, mas sensível ao atraso.
Taxa de recompra	~3%	Baixa: forte oportunidade de fidelização.

A história em uma frase

A operação vende bem, mas entrega devagar e fideliza pouco. Reduzir o prazo de entrega e atacar a recompra são as duas alavancas com maior efeito sobre receita e satisfação — e o dashboard foi desenhado para acompanhar exatamente essas duas frentes.

2. Definição do problema de negócio

2.1 Contexto

Assumimos o papel da equipe de BI da Olist. A empresa não vende produtos próprios: ela é o intermediário que coloca milhares de pequenos vendedores dentro dos grandes marketplaces e cuida da experiência de ponta a ponta — do anúncio à entrega e à avaliação. Por isso a Olist é cobrada por dois lados ao mesmo tempo: pelos vendedores, que querem vender mais, e pelos consumidores, que querem receber rápido e bem. Quando a entrega falha, a nota cai, o marketplace pressiona e o vendedor reclama.

2.2 Problema de negócio

Pergunta central

Onde a operação concentra os maiores problemas de prazo e satisfação, e quais regiões, categorias e vendedores deveriam ser priorizados para reduzir atrasos e aumentar a recompra dos clientes?

2.3 Stakeholders

Quem consome	O que precisa do dashboard
Diretoria	Visão executiva: receita, satisfação e prazo num relance, com evolução no tempo.
Gerência de operações/logística	Onde os atrasos se concentram (região, transportadora, período) para agir.
Equipe comercial	Quais categorias e vendedores puxam ou derrubam o resultado.
Relacionamento/CX	Relação entre atraso, nota e recompra, para priorizar quem reter.

2.4 Hipóteses iniciais

- O atraso na entrega é o principal fator que derruba a nota de avaliação.
- Pedidos com origem e destino em estados diferentes (especialmente fora do Sudeste) demoram mais.
- A recompra é baixa e fortemente influenciada pela experiência da primeira entrega.
- Poucas categorias concentram a maior parte da receita (cauda longa).

2.5 Critério de sucesso

O projeto entrega valor se, abrindo o dashboard, um gestor consegue em menos de um minuto: (1) ver se receita, prazo e nota estão melhorando ou piorando no período; (2) identificar as três regiões ou categorias que mais precisam de atenção; e (3) sustentar uma decisão concreta — por exemplo, priorizar a renegociação de prazos com transportadoras de uma região específica.

3. KPIs e OKRs

3.1 Indicadores-chave (KPIs)

Sete KPIs cobrem as três dimensões do problema — financeira, logística e de relacionamento. Cada um tem fórmula, unidade, meta e a razão de existir.

KPI	Fórmula	Unidade	Meta	Por que importa
Receita Total (GMV)	Σ preço dos itens	R\$	+10% YoY	Resultado financeiro central da operação.
Ticket médio	$\text{Receita} \div \text{n}^\circ \text{ de pedidos}$	R\$/pedido	$\geq$ R\$ 150	Mede quanto o cliente gasta por compra.
Prazo médio de entrega	$\text{média}(\text{data entrega} - \text{data compra})$	dias	$\leq$ 8 dias	Liga operação à satisfação.
Taxa de atraso	$\frac{\text{pedidos entregues após estimativa}}{\text{pedidos}}$	%	$\leq$ 5%	Atraso é o que mais derruba a nota.
Nota média	$\text{média}(\text{review_score})$	1 a 5	$\geq$ 4,3	Voz do cliente, sintetizada.
Taxa de recompra	$\frac{\text{clientes com 2+ pedidos}}{\text{clientes}}$	%	$\geq$ 6%	Fidelização custa menos que aquisição.
% frete sobre receita	$\Sigma \text{ frete} \div \text{Receita Total}$	%	$\leq$ 18%	Frete alto inibe conversão e recompra.

3.2 OKRs

Dois objetivos trimestrais, cada um com Key Results mensuráveis e ligados explicitamente aos KPIs acima.

OKR 1 — Tornar a entrega uma vantagem, não uma reclamação

KR1.1 — Reduzir o prazo médio de entrega de ~12 para 8 dias até o fim do trimestre. (KPI: prazo médio de entrega)

KR1.2 — Baixar a taxa de atraso de ~8% para 5%. (KPI: taxa de atraso)

KR1.3 — Elevar a nota média de ~4,0 para 4,3. (KPI: nota média)

OKR 2 — Vender mais para quem já é cliente

KR2.1 — Aumentar a taxa de recompra de ~3% para 6%. (KPI: taxa de recompra)

KR2.2 — Elevar o ticket médio de ~R\$ 137 para R\$ 150. (KPI: ticket médio)

KR2.3 — Crescer a receita em 10% ano a ano. (KPI: Receita Total)

O Objective dá a direção e não carrega número; os Key Results trazem a meta e o prazo, e cada um se apoia em um KPI que o dashboard calcula. Esse encadeamento problema → KPI → OKR é o que mantém o trabalho coerente do começo ao fim.

4. Modelagem dimensional (star schema)

Os dados da Olist vêm em oito tabelas separadas, no formato típico de exportação transacional. A modelagem reorganiza isso em um esquema analítico. Optamos por um esquema em constelação: dois fatos em granularidades diferentes, compartilhando dimensões conformadas. É uma escolha deliberada, justificada abaixo.

4.1 Por que dois fatos

Há dois eventos de negócio em granularidades distintas. A venda acontece no nível do item de pedido (um pedido pode ter vários produtos, de vários vendedores, cada um com seu preço e frete). Já a entrega, o pagamento e a avaliação acontecem no nível do pedido (uma nota, um prazo, um valor pago por pedido). Forçar tudo em um único fato obrigaria a repetir a nota e o prazo em cada item, inflando médias e contagens. Separar em dois fatos evita esse erro e mantém cada métrica na sua granularidade correta.

Tabela fato	Granularidade	Métricas (medidas de fato)
fato_itens	1 linha = 1 item de pedido	preço, valor do frete, quantidade (contagem de linhas).
fato_pedidos	1 linha = 1 pedido	valor pago, parcelas, nota da avaliação, tempo de entrega (dias), atraso vs. estimativa (dias).
Justificativa da granularidade Escolhemos o grão mais fino disponível em cada evento. Do item dá para agregar para pedido, categoria, vendedor ou região; do pedido dá para agregar para cliente e tempo. A regra: sempre é possível somar para cima, nunca desagregar o que já veio somado.		

4.2 Dimensões

Dimensão	Atributos principais
dCalendario	data (PK), ano, trimestre, mês (nome e número), dia, dia da semana, ano-mês. Construída em DAX, marcada como tabela de datas.
dCliente	id_cliente_unico (SK), cidade, estado (UF), região. Conformada — usada pelos dois fatos.
dProduto	id_produto (SK), categoria (em português), faixa de peso, faixa de volume. Liga apenas a fato_itens.
dVendedor	id_vendedor (SK), cidade, estado (UF), região. Liga apenas a fato_itens.

A dimensão Cliente merece uma nota: a tabela original tem um customer_id por pedido e um customer_unique_id que identifica a pessoa. Usamos o customer_unique_id como chave da dimensão, justamente para a taxa de recompra fazer sentido (senão todo cliente pareceria comprar uma vez só).

Geografia foi mantida dentro de Cliente e Vendedor, e não em uma dimensão separada, para evitar snowflake desnecessário.

4.3 Diagrama do modelo

Representação do esquema em constelação. O código DBML abaixo pode ser colado em dbdiagram.io para gerar a imagem do diagrama que vai em docs/modelo_dimensional.png.

```
DBML — colar em dbdiagram.io
Table dCalendario {
  data          date [pk]
  ano           int
  trimestre     int
  mes_num       int
  mes_nome      varchar
  ano_mes       varchar
}
Table dCliente {
  id_cliente_unico varchar [pk] // surrogate key da pessoa
  cidade          varchar
  uf              varchar
  regio          varchar
}
Table dProduto {
  id_produto     varchar [pk]
  categoria      varchar
  faixa_peso     varchar
}
Table dVendedor {
  id_vendedor    varchar [pk]
  cidade         varchar
  uf             varchar
  regio         varchar
}
Table fato_itens {
  id_pedido      varchar
  item           int
  id_produto     varchar [ref: > dProduto.id_produto]
  id_vendedor    varchar [ref: > dVendedor.id_vendedor]
  id_cliente_unico varchar [ref: > dCliente.id_cliente_unico]
  data_compra    date [ref: > dCalendario.data]
  preco         decimal
  frete         decimal
}
Table fato_pedidos {
  id_pedido      varchar [pk]
  id_cliente_unico varchar [ref: > dCliente.id_cliente_unico]
  data_compra    date [ref: > dCalendario.data]
  valor_pago     decimal
  parcelas       int
  review_score   int
  tempo_entrega  int
  atraso_dias    int
}
```

4.4 Relacionamentos

De (dimensão)	Para (fato)	Cardinalidade	Direção do filtro
dCalendario[data]	fato_itens / fato_pedidos	1:N	Única (dim → fato)
dCliente[id_cliente_unico]	fato_itens / fato_pedidos	1:N	Única (dim → fato)
dProduto[id_produto]	fato_itens	1:N	Única (dim → fato)
dVendedor[id_vendedor]	fato_itens	1:N	Única (dim → fato)

Decisões de modelagem (para a banca)

Tudo é 1:N com filtro de direção única — sem muitos-para-muitos e sem bidirecional, evitando ambiguidade de totais.

Calendário e Cliente são dimensões conformadas: os mesmos slicers de tempo e de região filtram os dois fatos ao mesmo tempo.

Nada de modelo flat (uma tabela gigante) nem snowflake puro: cada tabela tem um papel claro.

5. Transformações em Power Query

Todo o tratamento foi feito na camada de ETL do Power BI (Power Query / linguagem M). Cada etapa aplicada fica registrada e documentada. O objetivo é sair das oito tabelas brutas e chegar exatamente nas seis tabelas do modelo (dois fatos e quatro dimensões), já limpas e tipadas. Padrão de nomes adotado: snake_case, em português.

5.1 Transformações por tabela

Tabela de destino	Principais etapas
dCliente	A partir de customers: remover duplicatas por id_cliente_unico; juntar cidade/UF; criar coluna regiao via condicional a partir da UF (N, NE, CO, SE, S).
dProduto	A partir de products + tradução de categoria (merge com product_category_name_translation); preencher categoria nula com "não informada"; criar faixa_peso por intervalos.
dVendedor	A partir de sellers: tipar, padronizar UF, derivar regiao igual à de cliente.
dCalendario	Criada em DAX (Seção 6); marcada como tabela de datas no modelo.
fato_itens	A partir de order_items: merge com orders para trazer data_compra e id_cliente; tipar preco e frete como decimal; remover itens de pedidos cancelados.
fato_pedidos	A partir de orders: filtrar status "delivered"; calcular tempo_entrega e atraso_dias por diferença de datas; merge com reviews (nota) e com payments agregado por pedido (valor_pago, parcelas).

5.2 Checklist de qualidade aplicado

- **Tipagem:** datas como Data, valores como Número decimal, identificadores como Texto.
- **Nulos:** categorias nulas viram "não informada"; pedidos sem data de entrega ficam fora de fato_pedidos (não foram entregues).
- **Duplicatas:** removidas em dCliente (por id_cliente_unico) e nas dimensões de produto e vendedor.
- **Merge/Append:** merges para montar os fatos; agregação de pagamentos por pedido com Group By antes do merge.
- **Colunas calculadas de apoio:** regiao, faixa_peso, tempo_entrega, atraso_dias — criadas no ETL, não nos visuais.

Por que tratar no Power Query e não no visual

Manter a transformação no ETL deixa o modelo leve, reaproveitável e auditável. Coluna criada "na unha" dentro de um gráfico não é versionável nem reutilizável, e é onde os erros se escondem.

6. Medidas em DAX

São 17 medidas, todas em uma tabela de medidas dedicada (_Medidas), separada das tabelas de dados. A divisão atende e supera o mínimo do enunciado: medidas básicas, de inteligência de tempo e com CALCULATE + filtros. As mais complexas estão comentadas.

Dimensão Calendário (em DAX)

```
DAX – tabela de datas (marcar como tabela de datas)
dCalendario =
VAR _min = MIN( fato_pedidos[data_compra] )
VAR _max = MAX( fato_pedidos[data_compra] )
RETURN
ADDCOLUMNS(
    CALENDAR( _min, _max ),
    "ano",      YEAR( [Date] ),
    "trimestre", QUARTER( [Date] ),
    "mes_num",  MONTH( [Date] ),
    "mes_nome", FORMAT( [Date], "mmm" ),
    "ano_mes",  FORMAT( [Date], "yyyy-mm" )
)
```

6.1 Medidas básicas (8)

```
DAX
Receita Total      = SUM( fato_itens[preco] )
Receita Frete      = SUM( fato_itens[frete] )
Qtd Pedidos        = DISTINCTCOUNT( fato_pedidos[id_pedido] )
Qtd Itens           = COUNTROWS( fato_itens )
Clientes Únicos    = DISTINCTCOUNT( dCliente[id_cliente_unico] )
Ticket Médio       = DIVIDE( [Receita Total], [Qtd Pedidos] )
Nota Média         = AVERAGE( fato_pedidos[review_score] )
Prazo Médio (dias) = AVERAGE( fato_pedidos[tempo_entrega] )
```

6.2 Inteligência de tempo (4)

```
DAX
Receita YTD =
TOTALYTD( [Receita Total], dCalendario[Date] )

Receita Ano Anterior =
CALCULATE( [Receita Total], SAMEPERIODLASTYEAR( dCalendario[Date] ) )

Crescimento YoY % =
DIVIDE( [Receita Total] - [Receita Ano Anterior], [Receita Ano Anterior] )

Receita Média Móvel 3M = -- suaviza sazonalidade
AVERAGEX(
    DATESINPERIOD( dCalendario[Date], MAX( dCalendario[Date] ), -3, MONTH ),
    [Receita Total]
)
```

6.3 CALCULATE + filtros (5)

DAX

```
Pedidos Atrasados =      -- só pedidos entregues após a estimativa
CALCULATE( [Qtd Pedidos], fato_pedidos[atraso_dias] > 0 )

Taxa de Atraso % =
DIVIDE( [Pedidos Atrasados], [Qtd Pedidos] )

% Frete sobre Receita =
DIVIDE( [Receita Frete], [Receita Total] )

% Receita da Categoria =  -- participação no total, ignorando o filtro de produto
DIVIDE( [Receita Total], CALCULATE( [Receita Total], ALL( dProduto ) ) )

Taxa de Recompra % =      -- clientes com 2+ pedidos / total de clientes
VAR _recorrentes =
    CALCULATE(
        DISTINCTCOUNT( fato_pedidos[id_cliente_unico] ),
        FILTER(
            VALUES( dCliente[id_cliente_unico] ),
            [Qtd Pedidos] >= 2
        )
    )
RETURN DIVIDE( _recorrentes, [Clientes Únicos] )
```

Resumo do que foi atendido em DAX

8 medidas básicas (mín. 5), 4 de inteligência de tempo (mín. 3) e 5 com CALCULATE + filtros (mín. 4) — total de 17 medidas (mín. 12).

Todas na tabela _Medidas; as não triviais (média móvel, recompra) trazem comentário explicando a lógica.

7. Row-Level Security (RLS)

Três papéis simulam perfis reais de quem usa o dashboard. A segurança é aplicada por filtro DAX sobre a dimensão, e o filtro propaga para os dois fatos pelos relacionamentos. Implementação estática, suficiente para o cenário.

Papel (role)	Filtro DAX aplicado	O que enxerga
Diretoria	(sem filtro)	Todos os dados, todas as regiões.
Gerente Sudeste	dCliente[regiao] = "Sudeste"	Apenas pedidos de clientes do Sudeste.
Gerente Sul	dCliente[regiao] = "Sul"	Apenas pedidos de clientes do Sul.

7.1 Como foi testado

Usamos "Modelagem → Exibir como" (View as roles) no Power BI Desktop, selecionando cada papel e conferindo que os totais mudam conforme o esperado: ao entrar como Gerente Sul, a receita cai para a fatia do Sul e os filtros de outras regiões ficam vazios. Os prints de cada teste vão na documentação do repositório (docs/), como pede o enunciado.

Dica de banca

Na apresentação, deixe duas abas mentais prontas: entrar como Diretoria (vê tudo) e como Gerente Sul (vê só o Sul). A troca ao vivo, com os cartões de receita mudando na tela, é a prova visual de que o RLS funciona.

8. Dashboard

Três páginas com propósitos distintos, seguindo o fluxo do geral para o específico. Slicers de período (ano-mês), região e categoria ficam no topo, nomeados de forma clara, e acompanham o usuário entre as páginas.

8.1 Página 1 — Visão Executiva

- Faixa de cartões com os KPIs principais: Receita Total, Ticket Médio, Nota Média, Prazo Médio e Taxa de Atraso.
- Gráfico de linha da receita por ano-mês, com a medida de média móvel 3M sobreposta.
- Indicador de Crescimento YoY % com seta de tendência.
- Mapa ou barras de receita por região, para localizar onde está o volume.

8.2 Página 2 — Análise de Vendas e Produtos

- Barras de receita por categoria (Top 10) com % de participação no total.
- Matriz categoria × região com receita e ticket médio.
- Dispersão preço × frete por categoria, para ver onde o frete pesa mais.
- Cartão de % frete sobre receita, com o slicer de categoria ativo.

8.3 Página 3 — Logística e Satisfação

- Linha do prazo médio de entrega ao longo do tempo, comparado à meta de 8 dias.
- Barras da taxa de atraso por região e por faixa de prazo.
- Gráfico relacionando atraso médio e nota média (testa a hipótese central).
- Cartão de taxa de recompra, com recorte por região.

Boas práticas de design adotadas

Hierarquia visual: o que decide primeiro fica em cima e à esquerda. Cor de destaque só nos números que importam.

Slicers nomeados e posicionados de forma consistente nas três páginas.

Cada página responde a uma pergunta — Visão Executiva: "como estamos?"; Vendas: "o que vende?"; Logística: "onde dói?".

9. Versionamento e repositório

A entrega é em formato .pbip (não .pbix), porque o .pbip salva o relatório como arquivos de texto que o Git consegue versionar e comparar linha a linha. Os commits contam a história do projeto: "cria dimensões e calendário", "monta fatos no Power Query", "adiciona medidas de tempo", "implementa RLS por região".

9.1 Estrutura do repositório

```
.
├── README.md
├── docs/
│   ├── problema_de_negocio.md
│   ├── kpis_okrs.md
│   ├── modelo_dimensional.png
│   └── decisoes_de_modelagem.md
├── data/
│   └── README.md          # origem, link e licenca do dataset Olist
├── pbip/
│   └── ecommerceolist.pbip + pastas associadas
└── apresentacao/
    └── slides.pdf
```

9.2 Esboço do README

```
README.md
# Vendas, Logística e Satisfação no E-commerce (Olist)

Solução de BI ponta a ponta sobre o dataset público da Olist.

## Problema
Onde a operação concentra atrasos e baixa satisfação, e o que
priorizar para reduzir prazo de entrega e aumentar a recompra.

## Dataset
Brazilian E-Commerce Public Dataset by Olist (Kaggle) — ~100 mil
pedidos, 2016–2018. Link e licença em data/README.md.

## Principais KPIs
Receita (GMV), ticket médio, prazo médio de entrega, taxa de
atraso, nota média, taxa de recompra, % frete sobre receita.

## Como abrir
Power BI Desktop (versão recente) → Abrir → pbip/ecommerceolist.pbip

## Autor
Mateus Itibal
```

10. Pontos de revisão

10.1 Checklist final

Critério	Conferir	OK?
Problema de negócio	Pergunta única e clara, stakeholders, hipóteses e critério de sucesso (Seção 2).	☐
KPIs e OKRs	7 KPIs completos + 2 OKRs com KRs ligados aos KPIs (Seção 3).	☐
Modelagem	2 fatos + 4 dimensões + Calendário; granularidade justificada; diagrama gerado (Seção 4).	☐
Power Query	Tipagem, nulos, duplicatas e merges documentados (Seção 5).	☐
DAX	17 medidas na tabela _Medidas; complexas comentadas (Seção 6).	☐
RLS	3 papéis testados com "Exibir como" + prints (Seção 7).	☐
Dashboard	3 páginas com propósitos distintos e KPIs em destaque (Seção 8).	☐
Repositório	Projeto .pbip, README completo, commits significativos (Seção 9).	☐
Apresentação	10 min ensaiados, demo ao vivo testada, os dois falam.	☐

10.2 Perguntas prováveis da banca

Pergunta	Como responder
Por que dois fatos?	Venda é grão de item; entrega/nota é grão de pedido. Juntar inflaria médias. Dois fatos com dimensões conformadas mantêm cada métrica correta.
Por que customer_unique_id na dimensão Cliente?	Senão cada pedido viraria um cliente novo e a recompra daria ~0. O unique_id identifica a pessoa.
O que a medida de recompra faz?	Conta clientes com 2+ pedidos via FILTER sobre VALUES de cliente e divide pelo total de clientes.
Por que Calendário separado e em DAX?	Time intelligence exige tabela de datas contínua e marcada; a data do fato tem buracos.
Qual decisão o dashboard apoia?	Priorizar regiões/transportadoras para cortar atraso e atacar recompra — as duas alavancas de maior efeito.

Do problema à decisão: o dataset é real, o modelo é defensável e cada medida responde a uma pergunta de negócio. Resta construir o .pbip no Power BI Desktop seguindo este documento, versionar e ensaiar a demo.